

Università degli Studi di Urbino Carlo Bo
Corso di Laurea in Informatica – Scienza e Tecnologia
Programmazione e Modellazione a Oggetti, a.a. 2025/2026

Adventure Boy

Relazione per il Progetto del Corso di
Programmazione e Modellazione a Oggetti

Nam Polidori 334554

09/09/2026

Capitolo 1

Analisi

L'obiettivo Team è lo sviluppo di un gioco in stile Platform 2D, chiamato Adventure Boy.. Il gioco si sviluppa su più livelli suddivisi in mondi. L'obiettivo principale del giocatore è guidare il personaggio all'interno del livello superando gli ostacoli per raggiungere la porta finale e completare così l'intero percorso di ogni mondo per finire il gioco.

Durante l'esplorazione della mappa, il personaggio incontrerà dei nemici e potrà sconfiggerli utilizzando appositi oggetti collezionabili trovati nel corso del livello, i quali possono essere raccolti, conservati e gestiti tramite un apposito sistema di inventario.

L'applicazione offre inoltre un pannello di impostazioni che consente all'utente di personalizzare l'esperienza di gioco potendo configurare liberamente i tasti di movimento del personaggio e gestire la risoluzione e le modalità della finestra di gioco.

1.1 Requisiti

Requisiti funzionali

- Movimento del personaggio: L'applicazione deve permettere al giocatore di muovere il personaggio (salire, scorrere, saltare o avanzare) all'interno della mappa di gioco 2D.
- Completamento dei livelli e dei mondi: Il gioco deve prevedere una struttura a livelli e mondi; il livello si considera completato quando il personaggio raggiunge la porta finale, e il gioco è completato al superamento di tutti i livelli di tutti i mondi.
- Gestione dei nemici e combattimento: Nella mappa saranno presenti dei nemici che il personaggio potrà eliminare utilizzando gli oggetti collezionabili rinvenuti durante l'esplorazione.
- Inventario: L'applicazione deve includere un sistema di inventario che permetta al giocatore di raccogliere, custodire e selezionare gli oggetti collezionabili trovati nella mappa per utilizzarli contro i nemici.
- Pannello delle impostazioni: L'utente deve avere la possibilità di accedere a un menu di configurazione per:
 - Cambiare e rimappare i tasti di controllo per il movimento del personaggio.
 - Gestire le impostazioni della finestra di gioco.
 - Gestire e regolare i volumi e l'audio dell'applicazione.
- Salvataggio e caricamento dello stato: Il sistema deve permettere di tenere traccia dei progressi di gioco dell'utente (livelli completati e mondi sbloccati).

Requisiti non funzionali

- Reattività dei comandi: L'interazione da tastiera per il movimento del personaggio e la gestione dell'inventario deve garantire una risposta fluida e in tempo reale per non compromettere l'esperienza di gioco (Game Loop efficiente).
- Organizzazione delle risorse: La gestione di texture, sprite dei personaggi, nemici, oggetti e file di configurazione audio/video deve essere strutturata in modo standardizzato e modulare per facilitare l'estensione e la manutenzione.

1.2 Analisi e modello del dominio

Il gioco andrà a gestire il gameplay del giocatore.

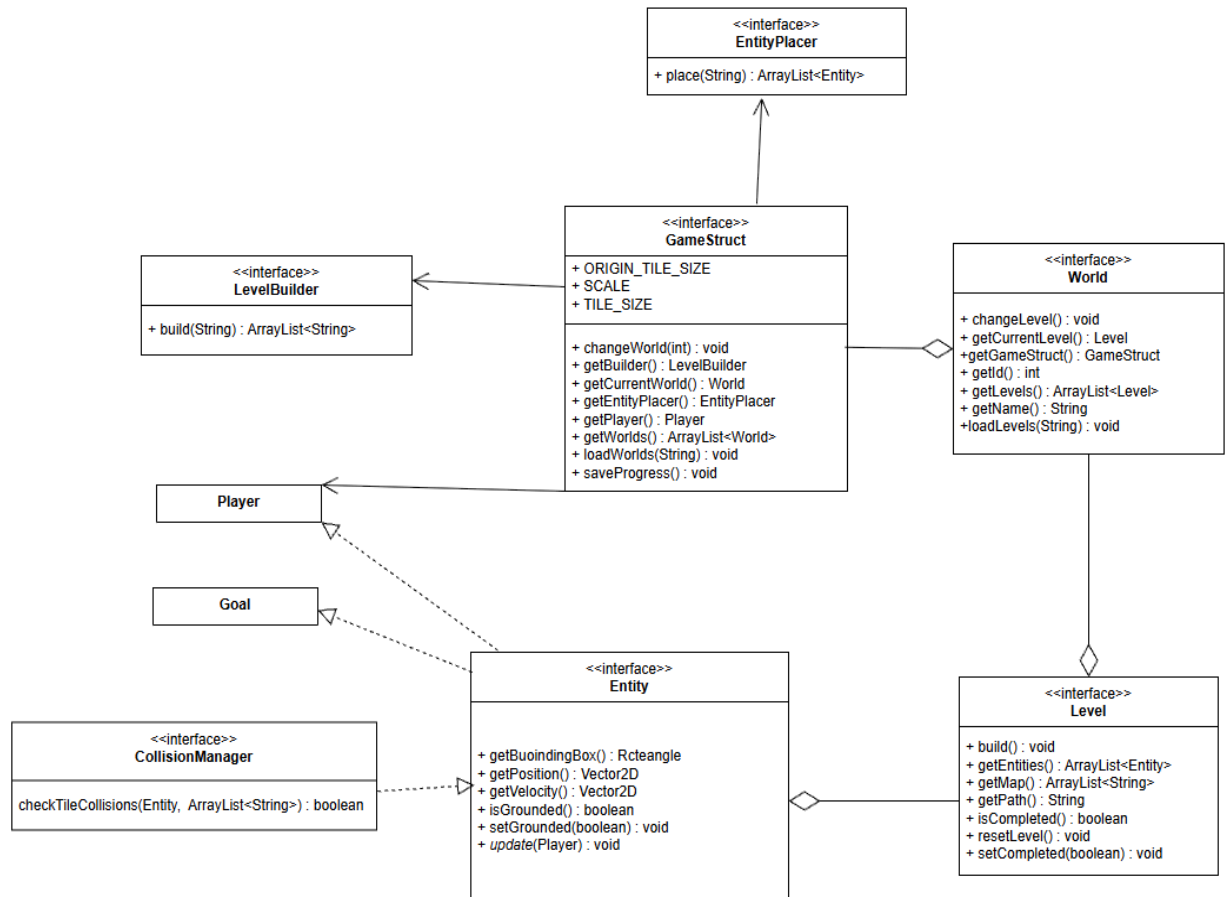
Il giocatore potrà scegliere il mondo e i vari livelli dei mondi generati da World e Level, dove ogni livello presenta percorsi diversi. Una volta scelto il livello il Giocatore utilizzerà il Personaggio (Player) per giocare e cercare di finire il livello.

Quindi il gioco per gestire la partita, dopo aver scelto il livello, avrà bisogno di diverse componenti e informazioni:

- Il giocatore identificato da Player.
- La mappa, generata da Level (Gerarchia GameStruct->World->Level e LeveBuilder)
- La generazione della porta finale (Goal)
- La generazione dei nemici e degli oggetti collezionabili / utilizzabili (EntityPlacer)
- Collisione del Player

Level builder si occuperà di leggere da file e costruire la mappa del livello scelto, mentre l'EntityPlacer si occuperà sempre di leggere da file ma di posizionare nella mappa le entità, come i nemici, oggetti collezionabili e oggetti neutri.

Il collisionManager invece si andrà ad occupare delle collisioni del Player e delle Entity e in base a quale Entity andrà a collidere il personaggio reagirà in modo diverso. Quindi se il personaggio collide con un nemico gli verrà detratta un po 'di vita, se il personaggio colliderà con un oggetto collezionabile lo prenderà.

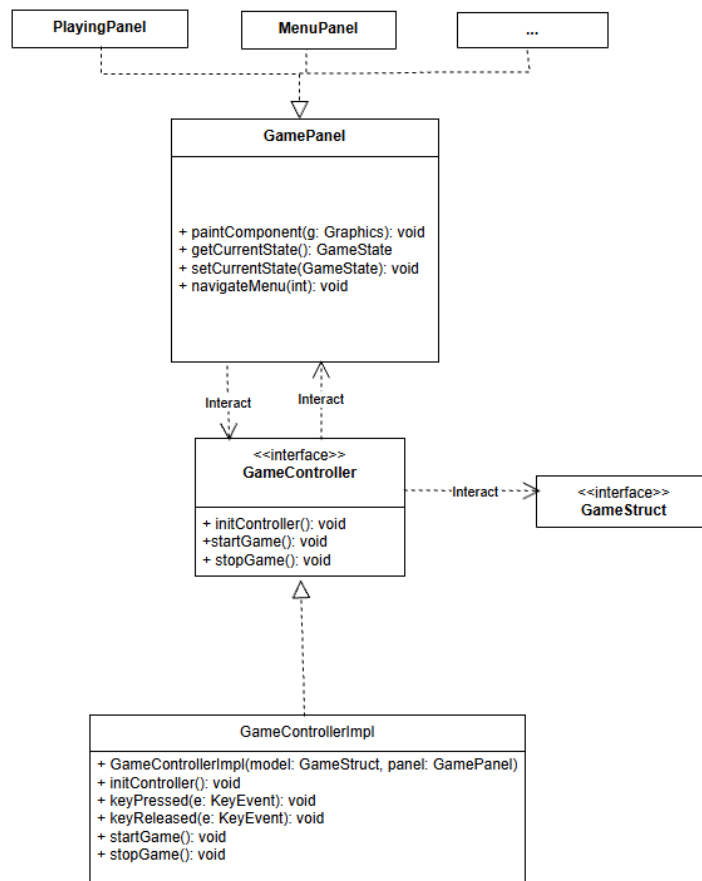


Capitolo 2

Design

2.1 Architettura

Per lo sviluppo del videogioco Platform 2D è stata adottata un'architettura modulare e a livelli, orientata alla separazione delle responsabilità tra la logica di gioco (Game Loop e Model), il sistema di rendering (View) e la gestione degli input e delle configurazioni (Controller), ispirandosi al pattern architetturale MVC (Model-View-Controller) o a pattern specifici per i game engine.



2.1.1 Model

Il Model gestisce i dati e lo stato logico del gioco. Il suo compito principale è aggiornare le informazioni (come la posizione del giocatore, i nemici o lo stato dei livelli) in base agli input ricevuti dal Controller, separando la logica di calcolo della parte grafica.

Per fare questo, il Model si appoggia all'interfaccia principale **GameStruct** (implementata dalla classe **GameStructImpl**), che funge da contenitore centrale per lo stato globale dell'applicazione. Al suo interno vengono gestiti il giocatore, i mondi di gioco e i livelli attivi.

Le componenti principali del Model:

- **Gestione di Mondi e Livelli (GameStruct, World, Level):** Rappresentano la struttura dei dati di gioco. Tramite classi di supporto come il **LevelBuilder**, il Model legge e carica i livelli, tenendo traccia del loro stato di completamento e della progressione del giocatore.
- **Gerarchia delle Entità (Entity, Object, Character, Player, Enemy):** Definiscono gli oggetti presenti nel mondo virtuale. Sfruttando l'ereditarietà basata sulla classe **Object** e sull'interfaccia **Entity** (insieme alla classe di supporto geometrico **Vector2D**), il sistema gestisce il movimento e le proprietà di tutti gli elementi attivi: il personaggio

(Player), i nemici (Enemy, ShootingEnemy), gli oggetti raccogliabili (Collectible) e i proiettili (Projectile).

- Gestione della Fisica e delle Collisioni (CollisionManager, EntityPlacer): Moduli di servizio dedicati al controllo spaziale. Il CollisionManager verifica in tempo reale se le entità collidono tra loro o con i muri, aggiornando lo stato dei dati in modo del tutto indipendente dai componenti della View.

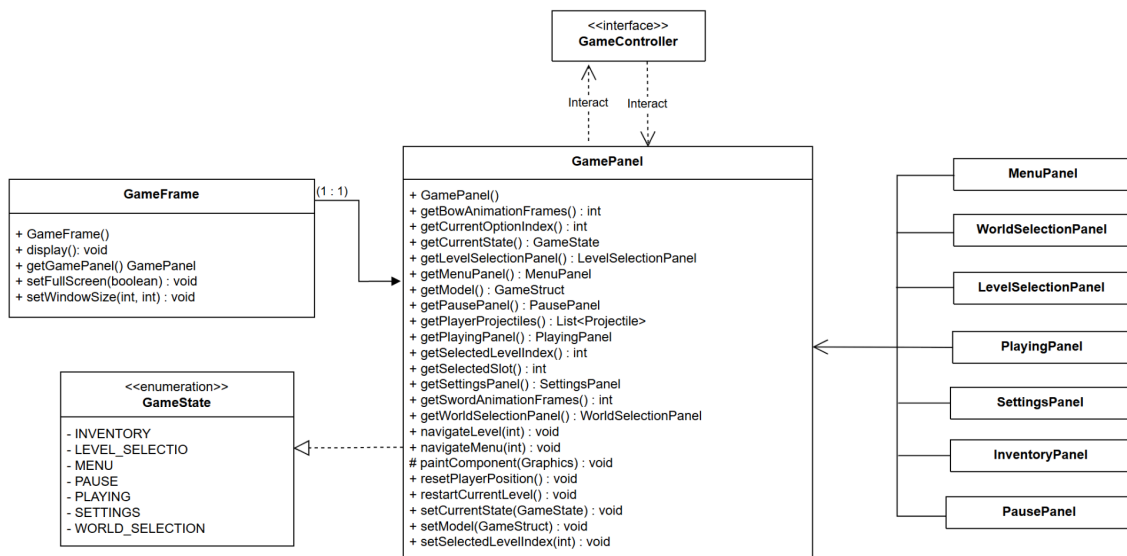
2.1.2 View

La View si occupa della gestione della parte grafica, del rendering visivo e della presentazione dello stato dell'applicazione all'utente. Il suo compito principale è mostrare visivamente i dati provenienti dal Model e rispecchiare i cambiamenti di stato gestiti dal Controller.

Nel tuo progetto, la struttura della View si basa su una finestra principale (GameFrame) che contiene e coordina il pannello di gioco centrale (GamePanel). Quest'ultimo gestisce lo stato corrente dell'applicazione (GameState) e funge da contenitore per le diverse schermate e sotto-pannelli specializzati dell'interfaccia.

Le componenti principali della View:

- Finestra e Pannello di Controllo Principale (GameFrame, GamePanel): Il GameFrame costituisce la finestra di sistema dell'applicazione, mentre il GamePanel è il componente grafico centrale che si occupa del ciclo di rendering principale (tramite il metodo paintComponent) e coordina la visualizzazione dei diversi contesti di gioco.
- Schermate e Pannelli di Interfaccia (MenuPanel, PlayingPanel, PausePanel, SettingsPanel): L'interfaccia utente è suddivisa in pannelli modulari dedicati a specifiche fasi o sezioni: il MenuPanel per l'avvio, il PlayingPanel per la sessione di gioco attiva, il PausePanel per la pausa e il SettingsPanel per la configurazione dei comandi e delle preferenze.
- Pannelli di Navigazione e Selezione (LevelSelectionPanel, WorldSelectionPanel, InventoryPanel): Moduli grafici di supporto che permettono all'utente di interagire con la progressione di gioco, consentendo di scegliere i mondi, i livelli da affrontare o di consultare l'inventario degli oggetti raccolti.



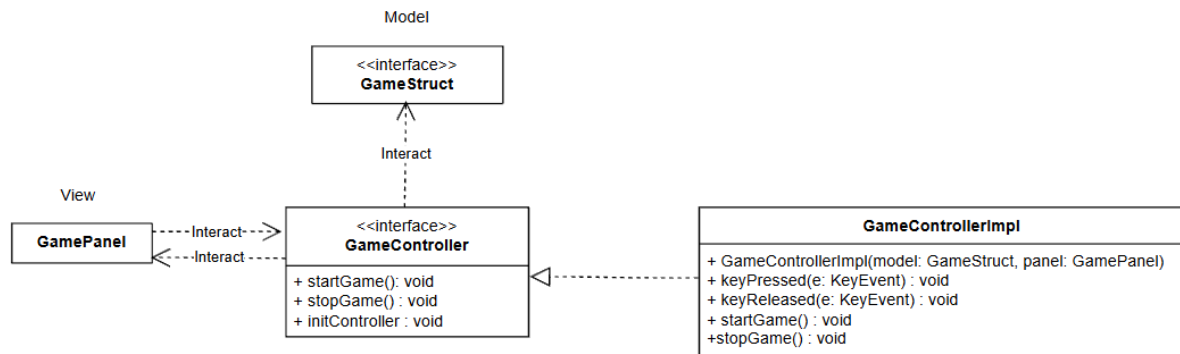
2.1.3 Controller

Il GameController è la componente cui spetta gestire in maniera consequenziale le interazioni da parte dell'utente nella View — nello specifico attraverso la gestione degli eventi di input da tastiera (keyPressed e keyReleased) — comunicando quindi al Model (GameStruct) il cambiamento avvenuto.

Una volta che il Model avrà completato l'elaborazione della richiesta di cambiamento, il controller avviserà la propria interfaccia grafica (panel), in modo tale che quest'ultima possa aggiornarsi in maniera coerente secondo le regole specificate.

Come si evince dalla struttura del package, si è scelto di adottare un'architettura basata su un'interfaccia (GameController) e una classe concreta di implementazione (GameControllerImpl), la quale incapsula i riferimenti diretti al model, al panel e allo stato precedente (previousState).

Da questa progettazione derivano le funzionalità chiave gestite dai metodi dedicati all'inizializzazione e al controllo del flusso di gioco (initController(), startGame() e stopGame()), evitando così la concentrazione di un'eccessiva responsabilità in un'unica classe.



2.2 Design dettagliato

Model

Struttura del gioco

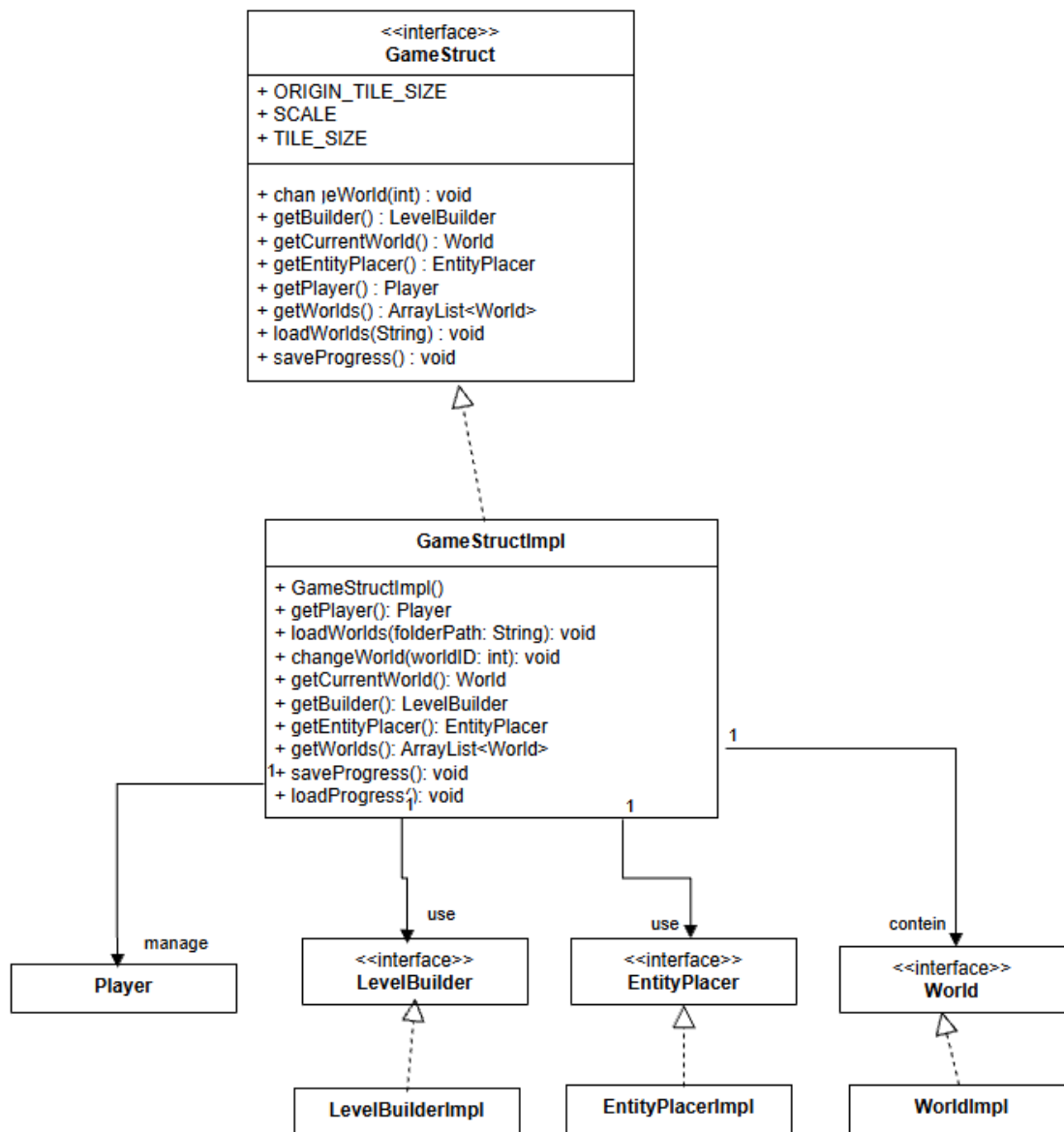
La componente software analizzata definisce il core model e la logica di controllo strutturale di un videogioco, occupandosi di coordinare i mondi, i livelli, il player e la persistenza dei dati attraverso il pattern basato sull'interfaccia `GameStruct` e la sua classe concreta `GameStructImpl`.

Da un lato, l'interfaccia stabilisce le specifiche architetturali e le costanti dimensionali di rendering basate su tile (`ORIGINAL_TILE_SIZE`, `SCALE`, `TILE_SIZE`), definendo il contratto per l'accesso alle entità e ai factory/builder di livello. Dall'altro, l'implementazione traduce questa struttura in logica eseguibile, gestendo l'inizializzazione del Player, del `LevelBuilderImpl` e dell' `EntityPlacerImpl` direttamente nel costruttore.

Il sistema implementa un caricamento dinamico dal file system (`loadWorlds`): scansiona la directory dei mondi, ordina alfabeticamente le sottocartelle e applica una manipolazione delle stringhe tramite regex per pulire i prefissi numerici e formattare i nomi delle istanze di `WorldImpl`. La gestione dello stato di esecuzione prevede il tracciamento del mondo attivo (`currentWorldIndex`) e metodi di navigazione e recupero delle collezioni tramite `ArrayList`.

Infine, la gestione della persistenza implementa un meccanismo di I/O su file di testo (`progress.txt`). La routine di salvataggio (`saveProgress`) effettua il parsing ricorsivo dei mondi e dei livelli filtrando quelli con flag di completamento attivo (`isCompleted()`), serializzandone i path univoci all'interno dello storage persistente.

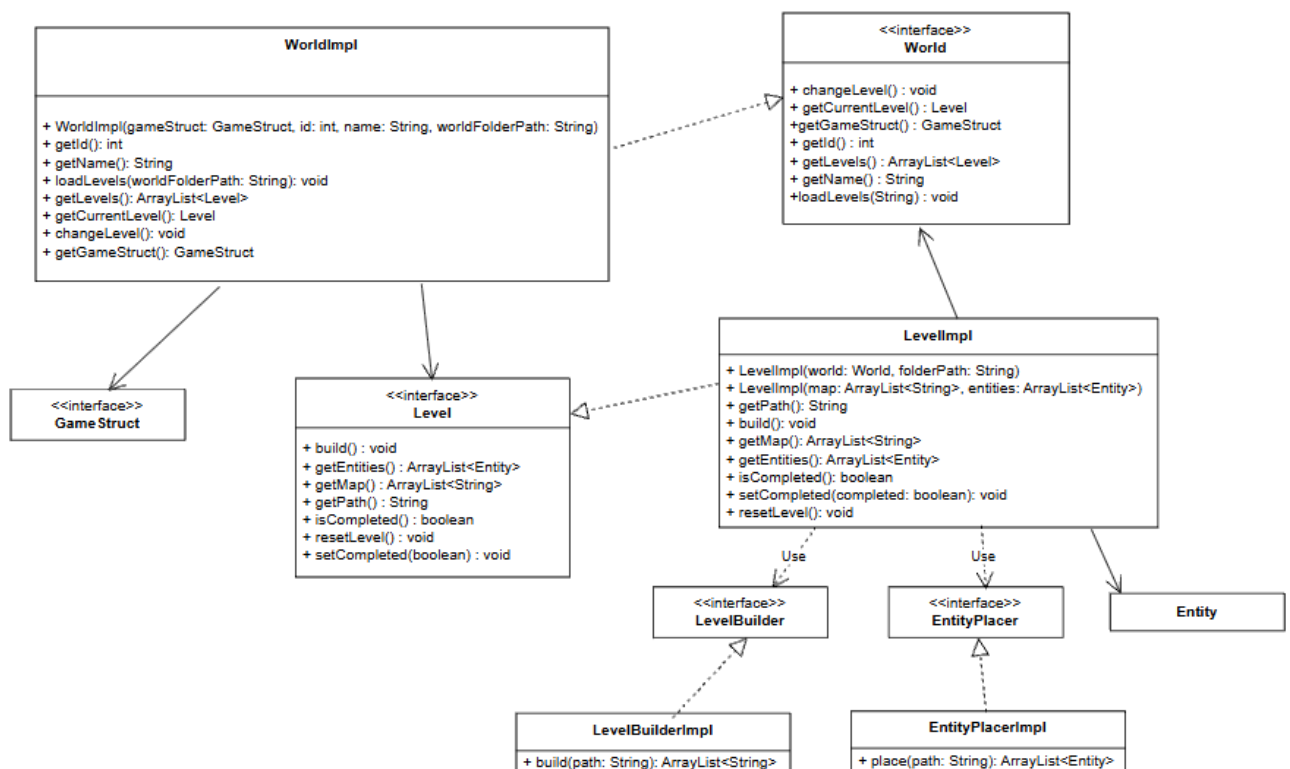
specularmente, il metodo loadProgress esegue il parsing del file all'avvio, deserializzando i path in una HashSet e ripristinando lo stato booleano dei livelli corrispondenti in memoria, garantendo così disaccoppiamento e scalabilità orizzontale dei contenuti di gioco.



World e Level

Continuando con la spiegazione del progetto, l'architettura si estende alla gestione gerarchica dei mondi e dei singoli livelli attraverso le interfacce World e Level e le rispettive classi di implementazione WorldImpl e LevelImpl, coadiuvate dal pattern di costruzione basato su LevelBuilder e LevelBuilderImpl.

Nello specifico, la classe WorldImpl gestisce una collezione ordinata di livelli leggendo le relative sottocartelle dal file system, consentendo di tracciare l'indice del livello attivo e di avanzare nella progressione di gioco tramite il metodo `changeLevel()`. Ciascun livello, rappresentato da LevelImpl, si occupa di inicializzarsi autonomamente andando a scansionare la propria cartella per individuare i file testuali della mappa e degli oggetti; per far ciò, sfrutta il LevelBuilderImpl per effettuare il parsing delle linee della mappa e un EntityPlacer per popolare la lista delle entità di gioco. Inoltre, la classe gestisce lo stato di completamento del livello tramite un flag booleano e mette a disposizione una routine di reset per ricaricare completamente i dati da zero in caso di necessità. Questa scomposizione modulare permette di disaccoppiare la logica di caricamento dei file I/O dalla struttura dei dati di gioco, rendendo l'aggiunta di nuovi mondi e livelli un'operazione scalabile e pulita



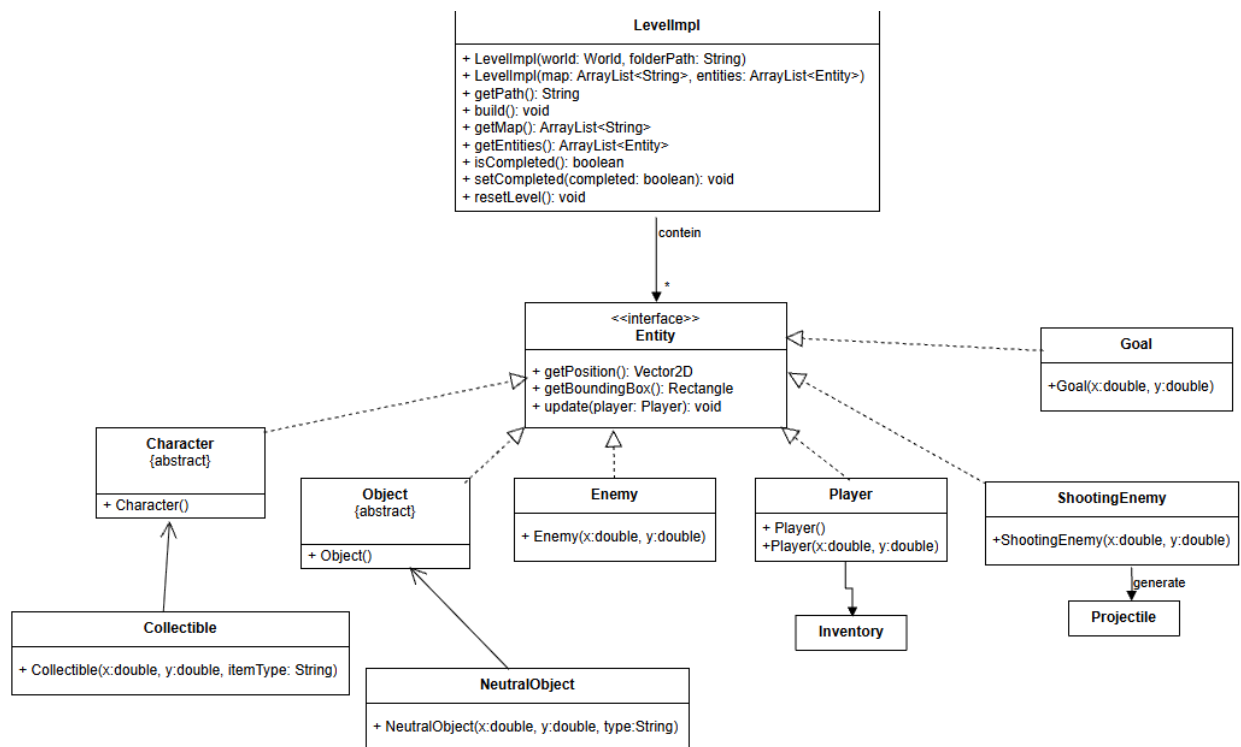
Oggetti ed Entità

Continuando, per quanto riguarda la modellazione delle entità di gioco, l'architettura si basa sull'interfaccia centrale Entity, la quale definisce il contratto comune per i bounding box di collisione, la fisica di base e i metodi di aggiornamento dello stato.

La gerarchia si dirama principalmente in due categorie: i personaggi e gli oggetti di mappa. Dal lato dei personaggi, troviamo la classe astratta Character che implementa le proprietà fisiche condivise, estesa concretamente dalla classe Player, la quale gestisce l'input di movimento, la gravità, i salti e lo stato vitale del giocatore, coadiuvato dal proprio Inventory. Sempre all'interno della categoria dei personaggi e delle minacce troviamo la classe Enemy e la sua specializzazione ShootingEnemy: queste gestiscono una logica di intelligenza artificiale basata su un raggio visivo, comportamenti di pattugliamento casuale, collisioni (con cooldown per infliggere danni al giocatore nel tempo e non azzerare la barra di vita in pochi frame), e, nel caso dello ShootingEnemy, la generazione e gestione dinamica di Projectile attivi.

Dal lato degli oggetti e degli elementi statici o interattivi, la classe astratta Object fa da tramite con l'interfaccia Entity per modellare elementi posizionati nel mondo di gioco. Tra questi troviamo la classe Collectible, che gestisce la logica di raccolta tramite collisione aggiungendo oggetti all'inventario, la classe NeutralObject per elementi di sfondo o non interattivi, e la classe Goal che funge da traguardo immobile per il completamento del livello.

Infine, a completare il quadro delle componenti di supporto, intervengono l'Inventory, i Projectile e la fondamentale classe geometrica Vector2D, utilizzata trasversalmente da tutte le entità per gestire con precisione matematica le coordinate spaziali, le posizioni e i vettori di velocità. Questa struttura orientata agli oggetti consente di mantenere un codice altamente estensibile, disaccoppiando la logica di aggiornamento e rendering delle singole entità dal motore di gioco principale.



Gestione delle collisioni

Il sottosistema di gestione delle collisioni e della fisica, governato dall'interfaccia `CollisionManager` e concretizzato dalla classe `CollisionManagerImpl`, costituisce il nucleo operativo per l'interazione spaziale all'interno dell'architettura di gioco. Tale componente si interfaccia direttamente con le entità dinamiche — tra cui il `Player` e le istanze di tipo `Enemy` — e con la rappresentazione matriciale della mappa, regolando la dinamica dei corpi e prevenendo la compenetrazione geometrica con gli elementi ambientali. Al fine di garantire prestazioni ottimali, l'algoritmo circoscrive la computazione ai soli blocchi adiacenti alla posizione corrente dell'entità, gestendo disgiuntamente lo spostamento sull'asse verticale (attuando il controllo della gravità, la gestione dei salti e il rilevamento dello stato di contatto con il suolo) e su quello orizzontale (delimitando i confini dello schermo e inibendo l'attraversamento di tile solidi). Tale modulo di controllo agisce inoltre da snodo logico per le interazioni di gioco avanzate: si coordina con le coordinate spaziali fornite dalla classe geometrica `Vector2D`, recepisce il rilevamento degli oggetti raccogliibili (`Collectible`) per l'aggiornamento dell'`Inventory`, gestisce le traiettorie dei `Projectile` e riconosce il contatto con l'elemento di traguardo (`Goal`), validando così il completamento dei livelli e mantenendo un rigoroso disaccoppiamento strutturale rispetto al motore di rendering principale.

View

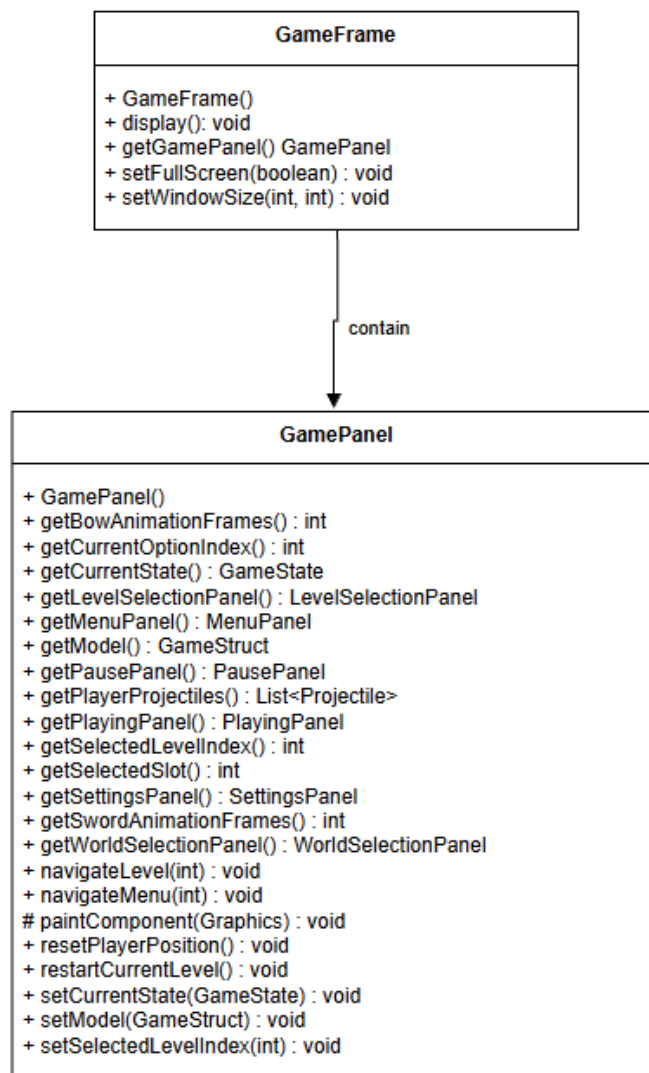
GameFrame

All'interno del package view, il contenitore principale dell'applicazione grafica è rappresentato dalla classe GameFrame, che estende JFrame per gestire la finestra intitolata "Hello Kitty Game". Architetturealmente, adotta un pattern di composizione incorporando un'istanza di GamePanel. La fase di inizializzazione si avvale di due costruttori: uno di default, che crea internamente un nuovo pannello, e un secondo che accetta un GamePanel come parametro per l'iniezione della dipendenza.

Entrambi configurano il titolo, l'operazione di chiusura predefinita (JFrame.EXIT_ON_CLOSE), disabilitano il ridimensionamento (setResizable(false)), aggiungono il pannello e applicano pack() con centratura dello schermo. Dal punto di vista funzionale, espone metodi quali display() per la visibilità, setWindowSize per modificare dinamicamente le dimensioni e setFullScreen per gestire la transizione a schermo intero tramite dispose(), setVisible(true), setUndecorated e JFrame.MAXIMIZED_BOTH.

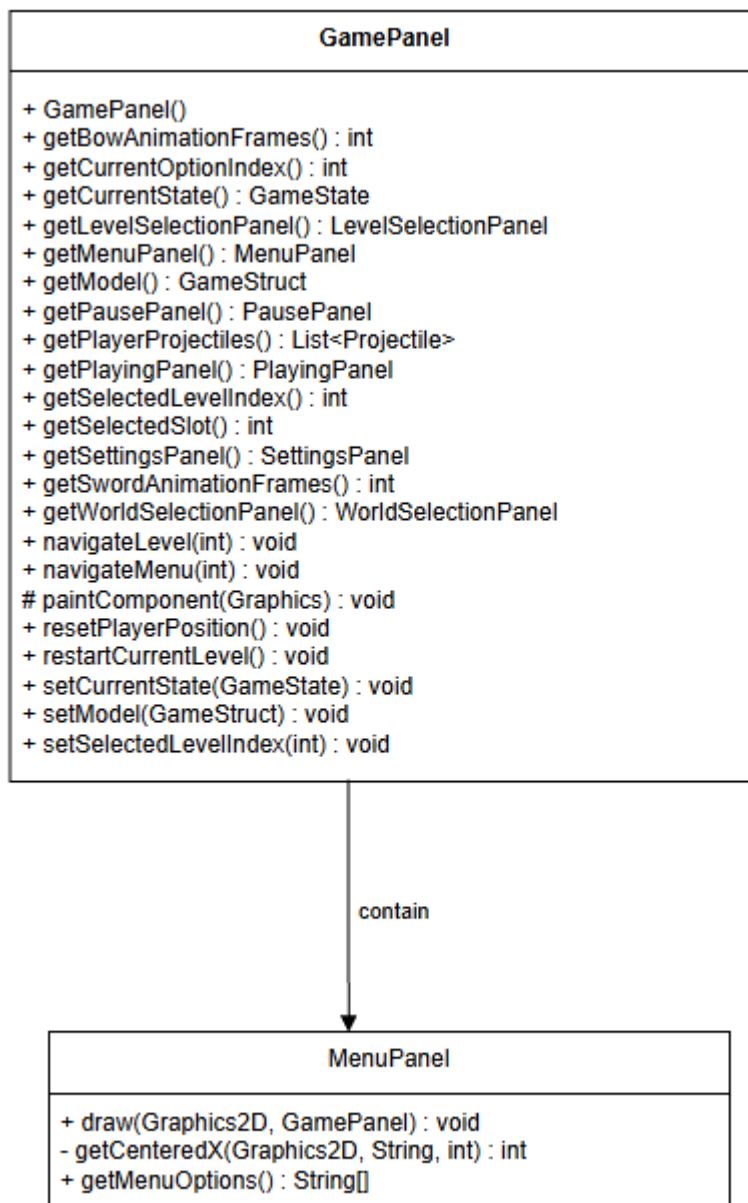
GamePanel (Schermata principale)

Operando come gestore di stati (State Controller) e coordinatore del ciclo di vita, la classe GamePanel (inclusa nel package view ed estendente JPanel) costituisce il nucleo centrale della gestione visiva e interattiva. Al suo interno coordina sotto-pannelli quali MenuPanel, SettingsPanel, WorldSelectionPanel, LevelSelectionPanel, PlayingPanel, PausePanel, InventoryPanel e il gestore delle collisioni (CollisionManagerImpl). Definisce dimensioni fisse e traccia lo stato corrente tramite l'enumerativo GameState. Il loop di gioco, basato su un Timer di Swing a intervalli regolari, aggiorna periodicamente la logica del giocatore, le collisioni, le animazioni e i nemici quando attivo. Gestisce inoltre l'input tramite listener dedicati per mouse, movimento e rotellina, implementando metodi di servizio per la navigazione, il ripristino della posizione e il rendering con l'override di paintComponent.



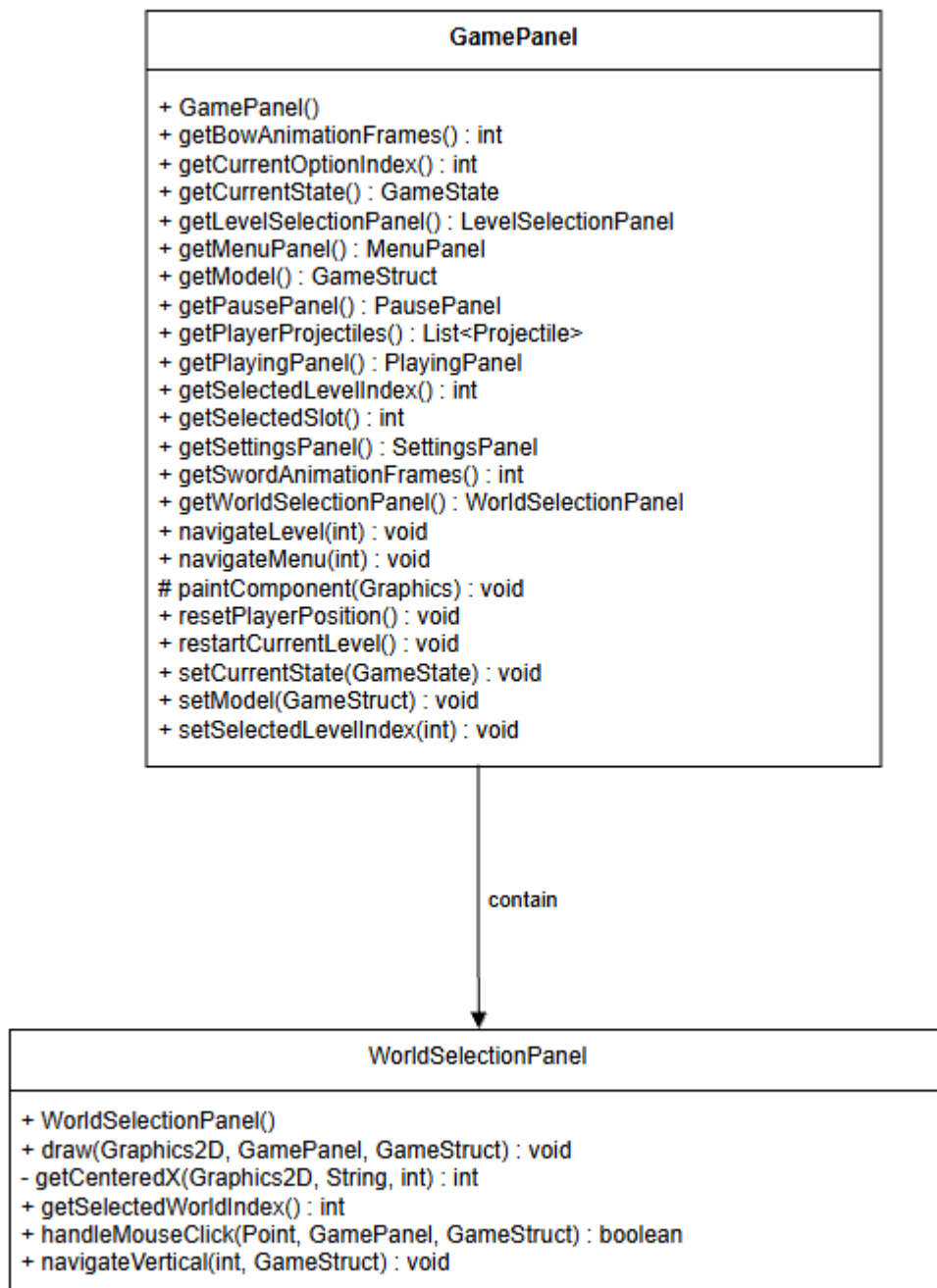
MenuPanel (Schermata di menù)

Il compito di gestire la rappresentazione grafica e la visualizzazione del menu principale dell'applicazione è affidato alla classe **MenuPanel**, situata nel package **view**. La classe definisce le opzioni di navigazione principali — avvio di una nuova partita, caricamento, impostazioni e chiusura del programma — occupandosi di disegnarle a schermo. Sfruttando il contesto grafico e coordinandosi con il pannello di controllo principale, evidenzia l'opzione attualmente selezionata mediante variazioni di colore e formattazione, calcolando il posizionamento centrato del testo in base alle dimensioni della finestra.



WorldSelectionPanel (Schermata di selezione dei mondi)

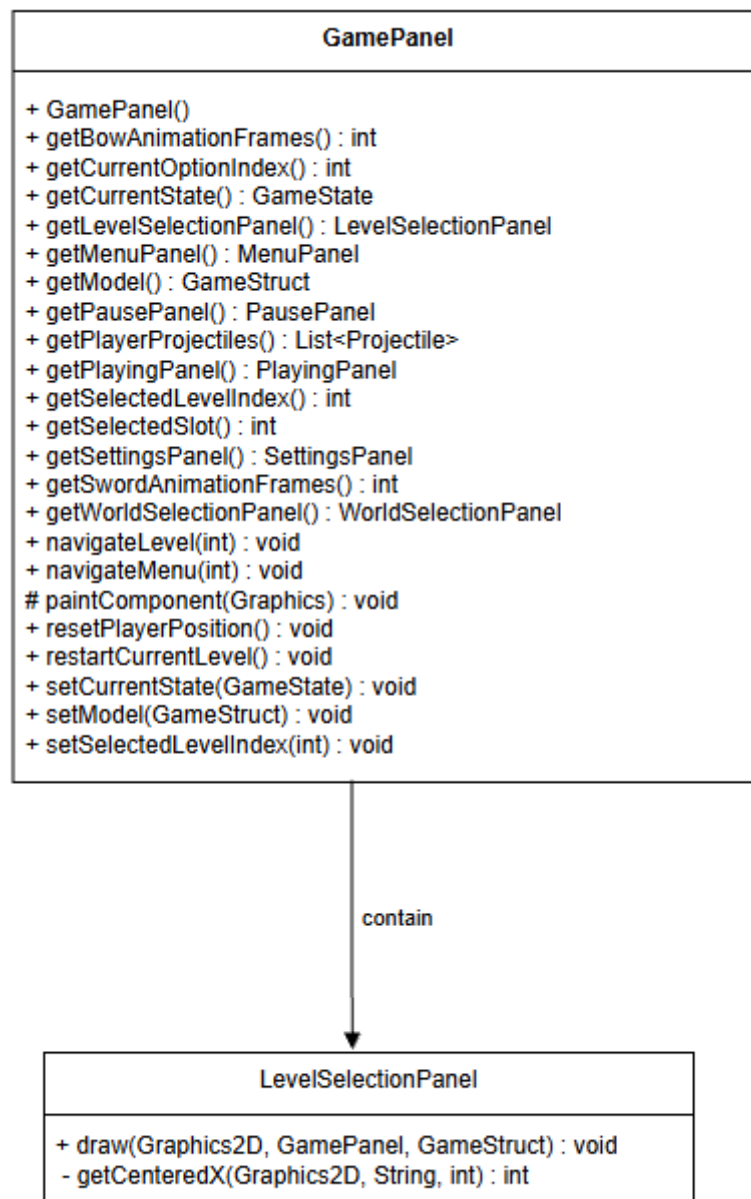
Per quanto riguarda la schermata e la logica di interazione per la selezione dei mondi di gioco, queste vengono gestite dalla classe **WorldSelectionPanel** del package **view**. La classe visualizza graficamente l'elenco dei mondi caricati dal modello e l'opzione per tornare al menu principale, calcolando la centratura e il posizionamento dinamico del testo. Gestisce inoltre lo stato di selezione consentendo la navigazione sia da tastiera che tramite mouse, aggiornando lo stato dell'applicazione e reindirizzando l'utente alla schermata di selezione dei livelli corrispondente.



LevelSelectionPanel (Schermata di selezione dei livelli)

Collocata nel package view, la classe LevelSelectionPanel si occupa della rappresentazione grafica e della visualizzazione della schermata di selezione dei livelli di un determinato mondo. Essa estrae e formatta informazioni strutturali e testuali direttamente dai file di configurazione, calcolando il posizionamento e la centratura del testo per il titolo del mondo, l'identificativo e il nome descrittivo del

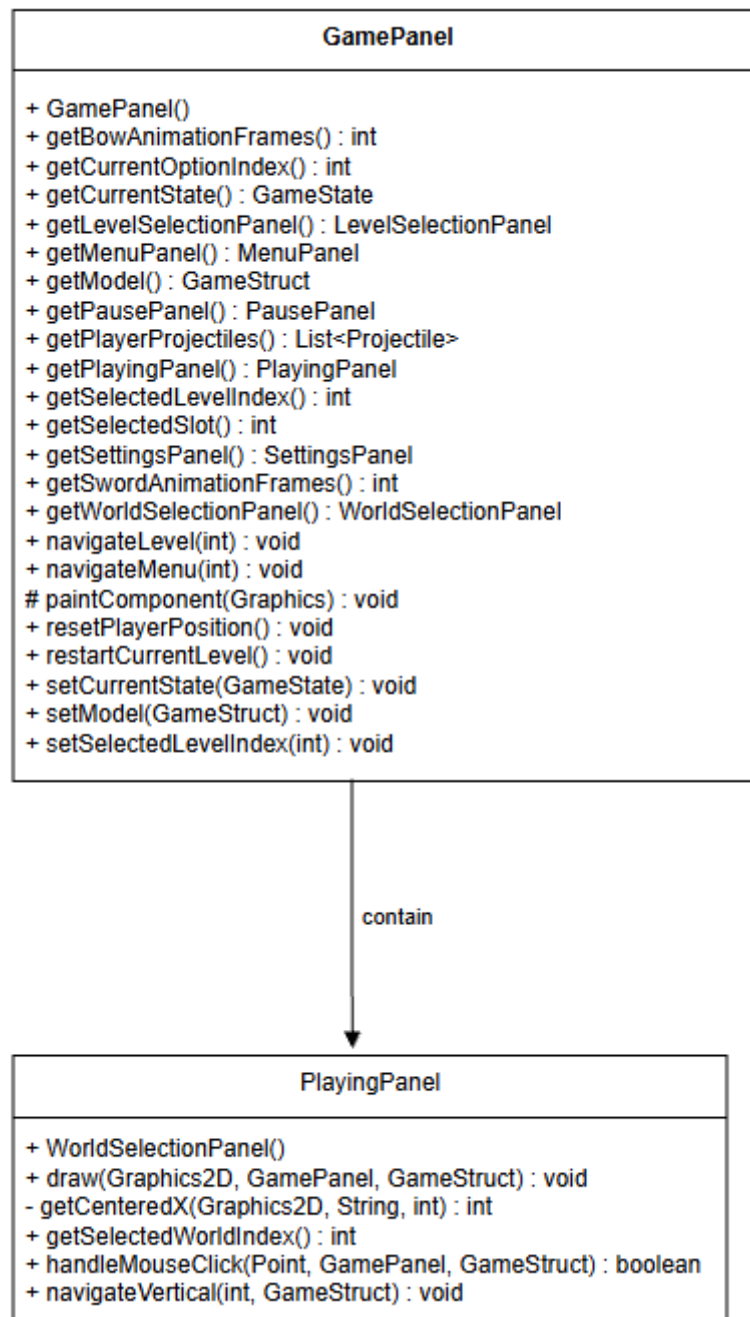
livello. Il pannello verifica inoltre lo stato di avanzamento del giocatore evidenziando visivamente il completamento del livello tramite indicatori dedicati a schermo.



PlayingPanel (Schermata di gioco)

L'intera gestione del rendering grafico e della rappresentazione visiva del gameplay attivo è affidata alla classe **PlayingPanel**, inclusa nel package **view**. Si occupa di caricare e gestire sprite dedicati per il personaggio, i nemici, gli oggetti collezionabili, gli elementi ambientali e le armi. Durante il disegno, applica una telecamera dinamica centrata sul giocatore per seguirne i movimenti nella mappa, visualizza tile di sfondo, entità e oggetti neutri, aggiorna gli elementi dell'HUD (salute e inventario

rapido) e gestisce effetti visivi per combattimenti, proiettili, completamento o reset in caso di sconfitta.

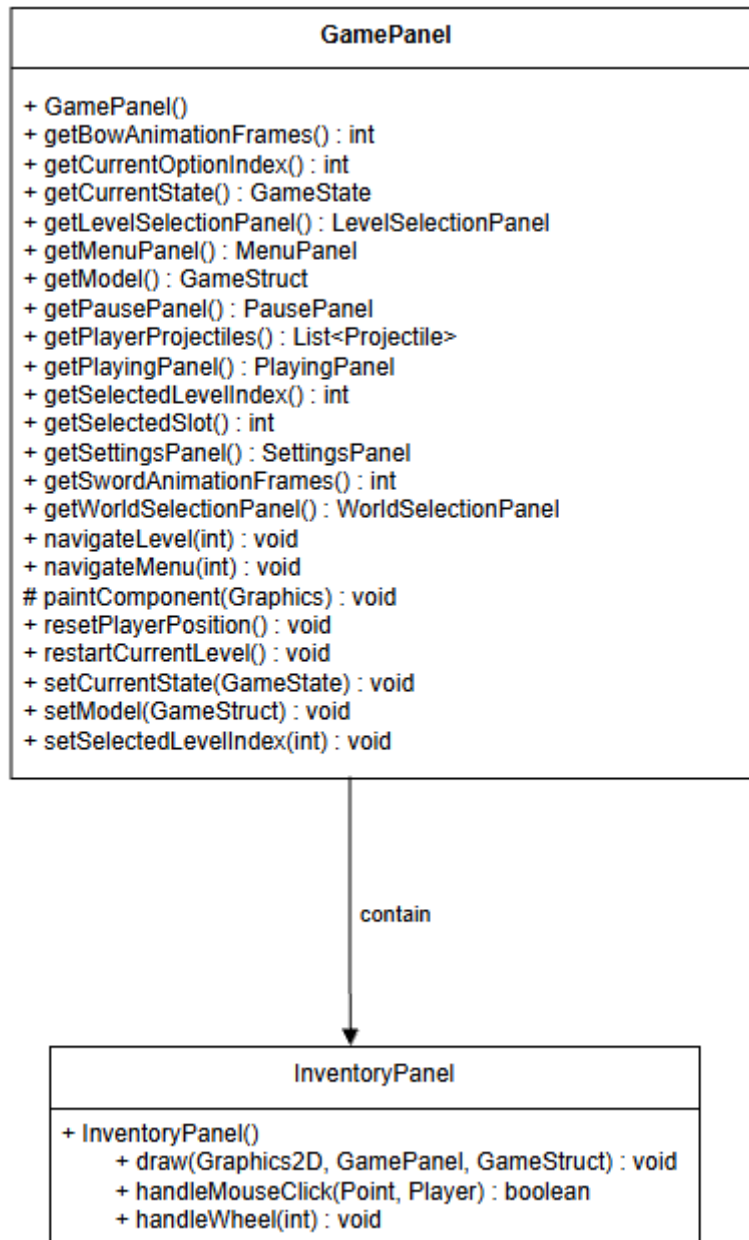


InventoryPanel (Schermata dell'inventario)

All'interno del package view troviamo la classe **InventoryPanel**, la quale gestisce la rappresentazione grafica e l'interazione utente per la schermata dell'inventario.

Carica le icone grafiche degli oggetti e visualizza la lista completa degli elementi posseduti dal personaggio, implementando un'area di scorrimento verticale

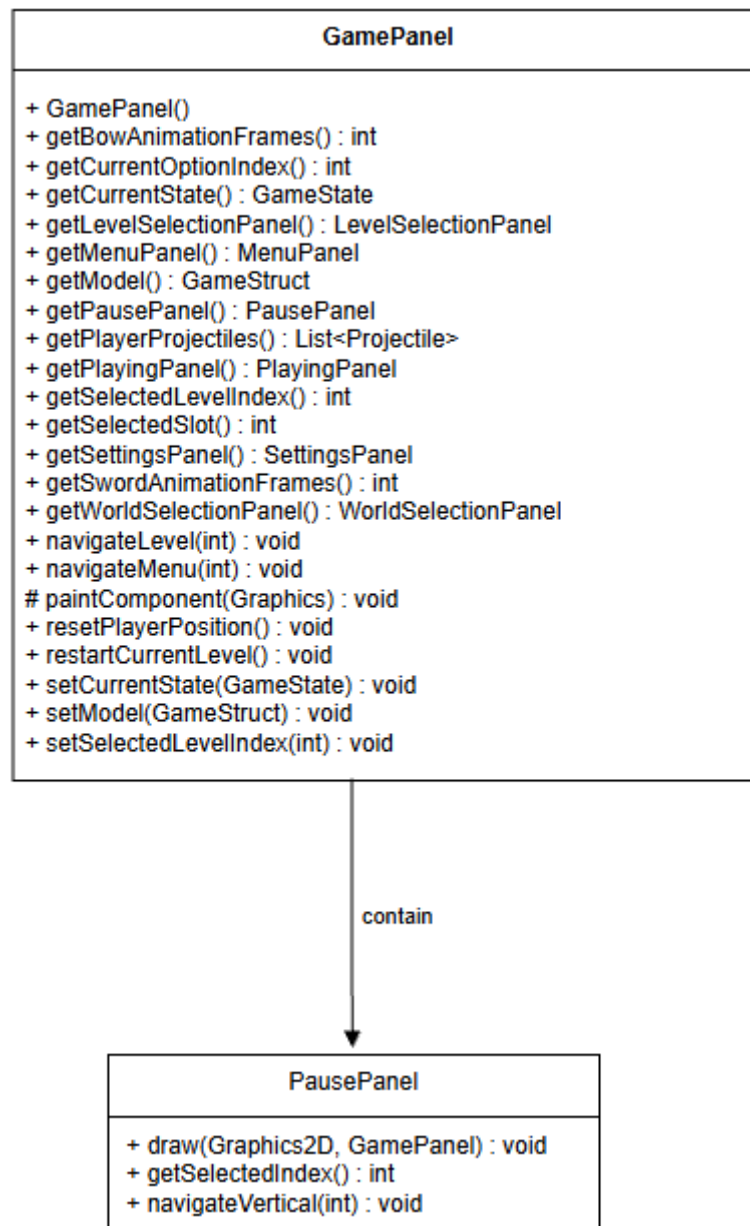
(scrolling) gestita con la rotellina del mouse. Gestisce inoltre l'interfaccia della barra rapida (hotbar), consentendo di selezionare e riorganizzare gli oggetti tramite mouse per associarli ai relativi slot di accesso rapido.



PausePanel (Schermata di pausa)

La rappresentazione grafica e la logica di navigazione del menu di pausa durante il gioco sono delegate alla classe **PausePanel**, inserita nel package **view**. Definisce le opzioni di controllo in stato di sospensione — ripresa, riavvio, impostazioni o ritorno alla selezione dei livelli — disegnandole a schermo con una sovrapposizione

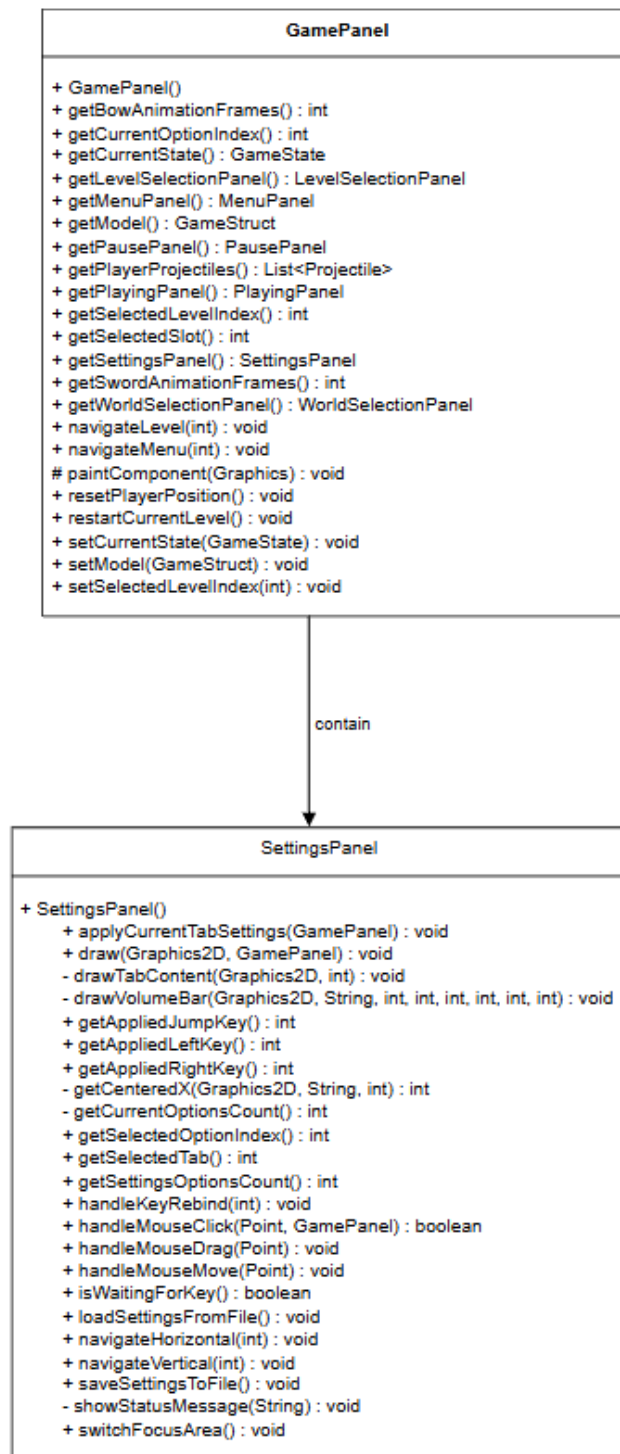
semitrasparente sullo sfondo. Gestisce lo stato di selezione tramite input verticale, evidenziando visivamente l'opzione scelta con variazioni di colore e formattazione basate sulle dimensioni dinamiche del pannello.



SettingsPanel (Schermata delle impostazioni)

Le preferenze di gioco e la relativa logica di configurazione — suddivise tra grafica e controlli — vengono gestite dalla classe **SettingsPanel** del package **view**. Visualizza e aggiorna parametri visivi quali risoluzione, limiti FPS, schermo intero e V-Sync, oltre a gestire la personalizzazione (keybinding) dei tasti di movimento e salto. Il pannello assicura inoltre la persistenza delle configurazioni salvandole e caricandole

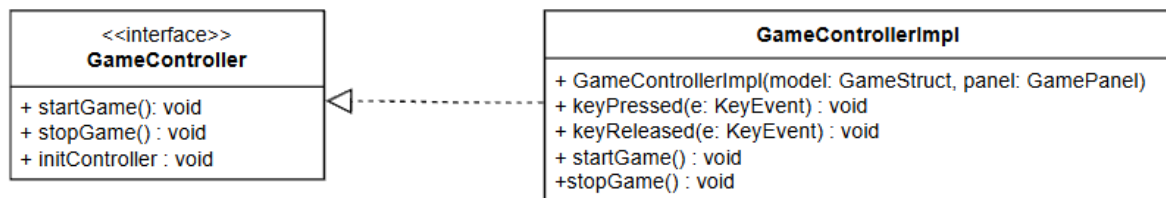
tramite file di testo dedicati, supportando sia la navigazione da tastiera che l'interazione tramite mouse.



Controller

Controller (GameController / GameControllerImpl)

Il sottosistema di controllo dell'architettura di gioco è costituito dall'interfaccia GameController e dalla sua implementazione GameControllerImpl, fungendo da raccordo tra gli input fisici da tastiera e lo stato logico-visivo di modello e viste. L'interfaccia estende java.awt.event.KeyListener e definisce i metodi fondamentali del ciclo di vita del controllo (initController, startGame, stopGame). La classe concreta estende KeyAdapter e gestisce centralmente pressione e rilascio dei tasti in base allo stato corrente (GameState): durante il PLAYING recupera i tasti personalizzati per il Player o gestisce interruzioni, mentre nei menu traduce i comandi per la navigazione, la modifica delle configurazioni e la transizione tra gli stati nel GamePanel.



3 Sviluppo

3.1 Testing Automatizzato

L'attività di testing è stata condotta adottando **JUnit 6** come framework di riferimento per la validazione automatizzata dei componenti core del *model*, effettuando i test sia in itinere, durante le fasi di implementazione delle singole feature, sia al completamento del progetto per prevenire regressioni.

Particolare attenzione è stata dedicata alla verifica dei moduli di I/O e alla gestione delle risorse persistenti, impiegando costrutti avanzati come l'isolamento tramite directory temporanee per testare il parsing delle mappe e il corretto posizionamento delle entità di gioco. L'implementazione di test mirati sulla logica di collisione ha inoltre garantito la stabilità del motore fisico e la robustezza complessiva del sistema.

Accanto a ciò, è stata eseguita un'intensa e ripetuta attività di testing non automatizzato sfruttando direttamente l'interfaccia grafica (GUI) del gioco. Questa fase di test empirici e funzionali è stata fondamentale per validare l'esperienza utente, la reattività dei comandi, il corretto rendering degli sprite e la fluidità delle transizioni tra i diversi menu e livelli in scenari d'uso reali.

Le principali suite di test automatizzati sviluppate comprendono:

- **LevelBuilderImplTest**
- **EntityPlacerImplTest**
- **CollisionManagerImplTest**
- **GameStructImplTest**

3.2 Metodologia di lavoro

Avendo sviluppato il progetto in completa autonomia, ho gestito in prima persona sia la parte di analisi e progettazione (che ha occupato circa la metà del tempo) sia l'implementazione effettiva, per un impegno complessivo stimato tra le 30 e le 40 ore.

Organizzazione del codice: Dato che il lavoro è individuale, l'intera struttura del progetto all'interno del package `game_hk` è stata creata da me. Per organizzare al meglio il codice, ho diviso le classi nei seguenti package principali:

- **Controller (controller):** Qui c'è la logica di controllo del gioco, gestita tramite GameController e GameControllerImpl.
- **Model (model):** È il cuore del gioco, dove ho inserito tutte le classi che riguardano le entità, le regole e la gestione del mondo di gioco. Tra queste ci sono:
 - La gestione dei livelli e della struttura di gioco (World, Level, LevelBuilder, GameStruct).
 - I personaggi, i nemici e il giocatore (Entity, Character, Player, Enemy, ShootingEnemy).
 - Gli elementi di supporto come collisioni, inventario e oggetti collezionabili (CollisionManager, Collectible, Inventory, Projectile, Vector2D).
- **View (view):** Raccoglie tutte le schermate e i componenti grafici dell'interfaccia utente, come la finestra di gioco (GameFrame), i pannelli principali (GamePanel, PlayingPanel), i menu, la pausa e le schermate di selezione dei livelli.

3.3 Note di sviluppo

Nam Polidori

- **Programmazione funzionale:** Ampiamente sfruttata durante lo sviluppo, in particolare per la gestione delle collezioni di entità, il filtraggio dei dati all'interno del motore di gioco e la manipolazione dei flussi di I/O. L'utilizzo di interfacce funzionali e di espressioni lambda ha reso il codice più dichiarativo, pulito e manutenibile, riducendo la verbosità specialmente nella gestione degli eventi e delle liste di elementi di gioco.
- **Java Swing / AWT:** Adoperato come framework grafico di base per la realizzazione di tutta l'interfaccia utente. Tramite l'estensione di componenti nativi come JFrame e JPanel, l'override del metodo paintComponent per il rendering personalizzato e l'utilizzo di Timer dedicati, è stato possibile implementare un game loop fluido e un sistema di pannelli modulari per la gestione dei menu, della schermata di gioco attiva e delle impostazioni.
- **Optional:** Utilizzati in modo estensivo in tutta la parte di model e nella gestione del caricamento dei file (come i file di mappa e configurazione). Si sono rivelati fondamentali per evitare il ritorno di valori null, prevenendo eccezioni a runtime e migliorando la chiarezza contrattuale dei metodi.
- **Gestione I/O e File di Testo:** Dovendo gestire il caricamento dinamico dei livelli, delle entità e della persistenza dei progressi tramite file di testo strutturati, ho implementato flussi di I/O dedicati (BufferedReader, FileReader, FileWriter) integrandoli con l'annotazione @TempDir di JUnit 6 per la scrittura di test robusti e isolati.

- Testing Automatizzato (JUnit 6) e Testing Grafico: L'adozione di JUnit 6 mi ha permesso di validare in modo incrementale i componenti critici del sistema (quali LevelBuilder, EntityPlacer e CollisionManager), affiancando ai test strutturati un'intensa e ripetuta attività di testing empirico condotta direttamente tramite l'interfaccia grafica (GUI) del gioco per testare la reattività e l'esperienza utente complessiva.